

TECHNICAL REVIEW & REDESIGN PROPOSAL

Pathfinder Web Scraping System

Chrome Extension Job Detection & Extraction Pipeline

Prepared for: Pathfinder engineering (extension/frontend surface)

Scope: `extension/src/content/*`, `extension/src/background/*`, extension-backend contract

Date: 2026-07-01

Method: direct source review of the current implementation, cross-checked against `pathfinder-tdd.md`

Executive Summary

The current scraping system is more capable than a typical MVP: `detect.js` already implements a weighted, multi-tier confidence score (URL/ATS patterns, JSON-LD, application-form sniffing, heading/body keywords, meta tags), and `scrape.js` already prefers structured `schema.org/JobPosting` JSON-LD over DOM scraping, falling back to a readability-style extractor with link-density scoring. Both are pure, side-effect-free, and the scraper has real unit tests. This is a solid foundation — better than "just grab `document.body.innerText`."

The system's largest weakness is not extraction quality on a static page — it is **staleness under dynamic content**. Detection and (to a lesser extent) scraping run once, at `document_idle`, against whatever DOM exists at that instant. There is no `MutationObserver`, no re-run on SPA client-side navigation (`pushState` /route changes), and no `all_frames` declaration for iframe-embedded ATS content. On a static server-rendered job page this is invisible; on LinkedIn Jobs, Workday, or any React/Vue/Angular careers site that fetches the description after first paint, it silently produces empty or wrong results.

A second finding worth flagging up front: `pathfinder-tdd.md` §6 describes a considerably larger system — per-ATS adapters, field-level autofill with synthetic input events, submission detection, a WASM on-page classifier.

None of this exists in code today. The built system only detects a job page and extracts text/JSON-LD for the backend to analyze. This review assesses what is actually built, and treats the TDD's autofill/adaptor vision as a roadmap input, not a baseline to compare against.

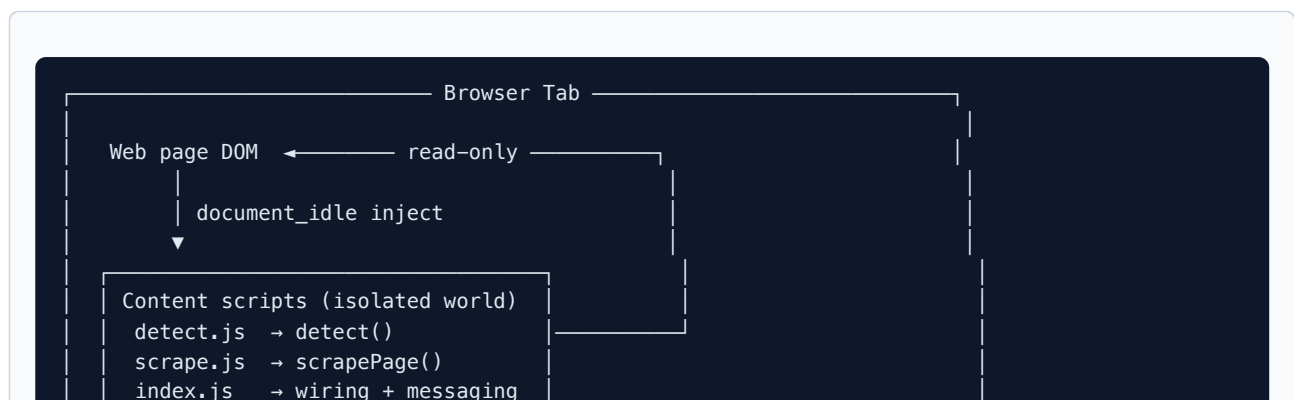
1. Current Architecture

1.1 Component Map

Manifest V3 extension, three run-time surfaces (`extension/manifest.json`):

- **Content scripts** (`src/content/detect.js`, `scrape.js`, `index.js`) — injected on `<all_urls>` at `document_idle`. Pure functions operating on the live DOM; no network calls, no `chrome.storage` access, no page mutation.
- **Service worker** (`src/background/index.js`, `api.js`, `storage.js`) — the only layer permitted to call the backend or touch durable storage. Non-persistent per MV3 (killed after ~30s idle); holds zero in-memory state, everything durable lives in `chrome.storage.session`, keyed by tab ID.
- **Side panel UI** (`src/sidepanel/*`) — the review surface. Talks to the service worker via `chrome.runtime.sendMessage` for detection/analysis/storage, and talks *directly* to the content script via `chrome.tabs.sendMessage` for on-demand scrapes (bypassing the service worker for that one path).

1.2 Architecture Diagram





1.3 Job Detection Process

`content/index.js` runs `detect()` exactly once, synchronously, as soon as the script is injected (`document_idle`), and fires-and-forgets a `PAGE_DETECTED` message to the service worker. The service worker persists the result to `chrome.storage.session` keyed by tab ID and sets the toolbar badge (`"JOB"`, blue) if `isJobPage`. There is no code path that re-runs detection later — the side panel's `GET_DETECTION` message only reads back whatever was cached at that one injection moment.

1.4 Job Extraction Process

Extraction is on-demand, not automatic: the side panel sends a `SCRAPE_PAGE` message directly to the content script (bypassing the service worker) whenever it needs fresh text — on open, on tab switch, and on `chrome.tabs.onUpdated` "complete". `scrape.js` then runs synchronously against the live DOM and returns a JSON payload (see §2).

1.5 Communication with the Backend

Only `background/api.js` calls the network. Base URL is read from `chrome.storage.local` (defaults to `http://localhost:8003`). Two endpoints matter to scraping: `POST /v1/jobs/analyze` (body: `{ raw_text, url }` — the scraped text is sent as an opaque string; the backend's `job_analysis_agent.py` does all structured re-extraction) and `POST /v1/resumes/generate` (keyed by the job id returned from analysis). `fetch()` calls have no `timeout/AbortController` and no `retry` — a hung backend hangs the calling UI action indefinitely with only the browser's own socket timeout as a backstop.

2. Current Scraping Strategy

2.1 What Is Actually Used

Technique	Used?	Detail
Generic DOM scraping (body text)	Fallback only	<code>pickMainContent()</code> falls back to <code>document.body</code> only if nothing scores above 0.
Site-specific selectors	No	No per-domain adapter map exists. One generic heuristic set (<code>CONTENT_SELECTORS</code>) is applied to every site.
Structured data (JSON-LD)	Yes, prioritized	<code>collectJobPostings()</code> parses every <code>script[type="application/ld+json"]</code> block, picks the <code>JobPosting</code> node with the longest description.
Readability-style scoring	Yes	<code>scoreElement()</code> : <code>text length × (1 - link density)</code> , ×1.5 for positive class/id hints, ×0.25 for negative hints.
TreeWalker text extraction	Yes	Walks the live DOM, skips hidden/noise subtrees via <code>FILTER_REJECT</code> , inserts line breaks at block boundaries.
OpenGraph metadata	Partial	Used only as a title/company backfill (<code>og:title</code> , <code>og:site_name</code>), not for detection confidence beyond <code>og:type</code> .

2.2 Assumptions Baked Into the Implementation

- The DOM at `document_idle` already contains the job content — true for server-rendered pages, false for many SPAs.
- A JSON-LD `JobPosting` description, once present, is trustworthy above 200 characters (`MIN_JSONLD_DESCRIPTION`) — reasonable, but not validated against the visible DOM (a stale/mismatched JSON-LD block would be trusted silently).
- Class/id substring hints (`job-description`, `posting`, etc.) generalize across arbitrary sites — true often enough to be useful, but this is guesswork, not per-site knowledge.
- Everything relevant lives in the top-level document — no shadow roots, no iframes.
- One page = one job posting — `collectJobPostings()` already handles *multiple* `JobPosting` nodes by picking the richest, but the readable-extraction fallback picks a single "best" container and cannot represent "two job descriptions on one page."

2.3 Strengths

- Structured-data-first ordering means the common case (ATS with SEO-friendly JSON-LD) costs zero DOM heuristics and is highly accurate.
- No page mutation, no cloning — avoids the classic "detached-clone `innerText` is empty" bug class (explicitly called out in the file's own header comment as a fixed prior issue).
- Defensive bounds: 500-element scoring cap, 30k-char truncation, malformed-JSON-LD is swallowed per-block rather than aborting the whole extraction.
- Pure functions with real jsdom unit tests (`tests/scrape.test.mjs`) covering JSON-LD, multi-posting selection, noise stripping, hidden-content skipping, and truncation.

2.4 Weaknesses

- Single point-in-time snapshot — no awareness that the DOM might still be loading or might change after extraction.

- No validation that extracted `text` is actually job-shaped (no length floor, no keyword sanity check) — a badly-scored container can return garbage with no signal to the caller that quality was low.
- No shadow-DOM or iframe traversal (see §3).
- Zero site-specific knowledge — every ATS is treated identically, so a layout that defeats the generic heuristics degrades silently to "body fallback" with no per-site remediation path.

3. Robustness Review

Table below is ordered by estimated real-world frequency for this product's actual traffic (major ATS platforms + LinkedIn/Indeed/Glassdoor + generic careers pages), not by theoretical exhaustiveness.

Scenario	Why it fails today	Frequency	Recommendation
SPA client-side navigation (LinkedIn Jobs, Greenhouse embedded SPA)	Content scripts inject once per document load. <code>pushState</code> navigation never reloads the document, so <code>detect.js</code> / <code>index.js</code> never re-run. The side panel's own re-scrape triggers (<code>tabs.onActivated</code> , <code>tabs.onUpdated</code> "complete") also don't fire on same-tab SPA route changes.	High	Content script installs a <code>history.pushState</code> / <code>popstate</code> patch or listens via a lightweight URL-poll (<code>location.href</code> diff) to re-run <code>detect()</code> and notify the service worker on route change.
Lazy/async-loaded job description (React/Vue/Angular fetch-after-mount)	<code>detect()</code> and <code>scrapePage()</code> run against whatever exists at <code>document_idle</code> . If the description streams in after an XHR/fetch completes post-paint, both can run before the content exists.	High	Add a bounded <code>MutationObserver</code> that re-scores content once meaningful DOM growth is observed (e.g., debounce 300–500ms after last mutation, cap total wait ~3s), rather than a single fixed-timing read.
Iframe-embedded ATS content (Lever, some Workday embeds)	<code>manifest.json</code> <code>content_scripts</code> do not set <code>"all_frames": true</code> ; <code>scrape.js</code> / <code>detect.js</code> only ever see the top document.	Medium	Add <code>all_frames: true</code> and have each frame report independently; service worker merges/picks the frame with the highest-confidence detection.
Shadow DOM (web-component-based ATS UIs)	<code>querySelectorAll</code> (used throughout <code>NOISE_SELECTORS</code> , <code>CONTENT_SELECTORS</code> , JSON-LD lookup) does not pierce shadow roots; the <code>TreeWalker</code> likewise stops at a shadow host.	Low today, rising	Recursive shadow-root walker: when a node has <code>shadowRoot</code> (open mode only — closed shadow roots are unreachable by design), descend into it during both selector queries and the <code>TreeWalker</code> pass.
Infinite scroll / paginated descriptions	Not applicable to single-job pages in practice; would matter if scraping job-list pages, which this pipeline does not target.	Low (out of scope)	No action — flag as explicitly out of scope for a single-posting extractor.
Cookie/consent banners	Handled reasonably: <code>NOISE_SELECTORS</code> strips <code>[class*='cookie']</code> / <code>[class*='consent']</code> from extracted text. Risk is cosmetic (a visible overlay) not a data-quality bug, since extraction is DOM-based, not screenshot-based.	Low	No change needed for text extraction; irrelevant to any future screenshot/visual-model path.
Multiple job postings on one page	JSON-LD path already handles this (picks richest description among several <code>JobPosting</code> nodes). Readable-extraction fallback does not — it commits to a single "best" container.	Low	Low priority: most multi-posting pages (list pages) aren't the extension's target; only worth fixing if a specific ATS pattern surfaces it in testing.
Very large job descriptions	Handled: 30k-char cap with word-boundary truncation and a <code>truncated / length</code> flag surfaced to the caller.	Low	No change needed; verify the backend doesn't silently drop the "truncated" signal when scoring ATS match.
Hidden content (<code>display:none</code> ,	Handled: <code>isHidden()</code> checked in both the <code>TreeWalker</code> (<code>FILTER_REJECT</code>) and	Low	Consider also excluding <code>clip-path</code> / off-screen-positioned "visually

visibility:hidden, opacity:0)	scoreElement() .		hidden" patterns used for a11y-only text, which can otherwise pollute extracted text with skip-link/screen-reader-only phrases.
International job boards (non-English keywords)	JOB_HEADING_KEYWORDS / JOB_BODY_KEYWORDS are English-only. A non-English posting with no JSON-LD and no ATS URL match can score 0 on Tiers 2c/2d and fall below the 0.4 detection threshold.	Medium (depends on target markets)	JSON-LD/ATS-URL tiers are language-agnostic and already carry most of the weight (0.6/0.5 + 0.4) — good baseline. Add a locale-keyword table only if/when non-English markets become a priority; don't over-invest speculatively.
Pages requiring login (gated job boards)	Out of scope by design — the extension operates on whatever the logged-in user's browser already renders; if content is behind a login the user hasn't completed, there is no DOM to read. Not a scraper bug.	Low	No action; document as a known limitation, not a defect.
Accessibility markup (ARIA) as a signal	Used defensively ([role='navigation'] etc. treated as noise) but never used positively — [role='main'] is in CONTENT_SELECTORS , but role="article" / aria-label hints on description containers aren't leveraged.	Low	Minor: add [aria-label*='job description'] -style selectors to CONTENT_SELECTORS ; cheap, low-risk addition.

4. Job Detection Review

4.1 What's Already Production-Grade

The requested "better approaches" in the task brief are, in fact, already implemented:

- **URL heuristics** — `ATS_URL_PATTERNS` , weighted 0.5–0.6 per platform.
- **Schema.org JobPosting / JSON-LD** — `hasJobPostingJsonLd()` , weight +0.4.
- **Open Graph metadata** — `hasJobMetaTags()` , weight +0.1.
- **Confidence scoring** — additive weighted score capped at 1.0, threshold 0.4, full signal breakdown returned to the caller (used to render the ✓/— signal grid in the side panel).
- **Multi-stage/escalating detection** — URL (instant) → structured data/forms/headings/body/meta (DOM-dependent), matching the tiered design described in the TDD.

What's *not* implemented from the TDD's wishlist: a "Tier 3" ML/WASM classifier for ambiguous cases (reasonable to defer — Tiers 1–2 already carry six independent signals, which is a lot of coverage for the added complexity a classifier brings), and per-site adapters (see §5).

4.2 Weaknesses Specific to Detection (not extraction)

Issue	Impact
Single-shot execution, no re-run trigger	The most consequential gap (§3). A SPA that renders its job content 500ms after <code>document_idle</code> will detect as "not a job page" permanently for that tab, and the badge will never update — no code path re-invokes <code>detect()</code> later.
Threshold (0.4) is a single unexplained magic number	Not wrong, but untested against a labeled corpus — there's no fixture set proving 0.4 minimizes false positives/negatives across real ATS pages vs. e.g. 0.35 or 0.5. This is a tuning parameter with no evaluation harness behind it.
No negative/false-positive corpus in tests	<code>detect.js</code> currently has <i>zero</i> unit tests (confirmed: no <code>detect.test.mjs</code> in <code>extension/tests/</code>), unlike <code>scrape.js</code> . The highest-leverage, most-tunable piece of the pipeline is the least tested.
<code>hasApplicationForm()</code> can false-positive on non-job forms	Any 3+ field form with a name/email/phone-shaped field (common on newsletter signup, contact-us, account-creation pages) can contribute +0.25 toward the threshold. Combined with a coincidental heading match this could tip an unrelated page over 0.4.

4.3 Recommended Production Detection Strategy

1. **Keep the existing tiered/weighted design** — it already matches best practice (URL → structured data → DOM heuristics, cheapest/highest-confidence signals first). Don't rebuild it.
2. **Add re-evaluation triggers:** a `MutationObserver` (debounced) plus a `pushState / popstate` hook that re-runs `detect()` and sends an updated `PAGE_DETECTED` — this single change fixes the largest real-world failure mode.
3. **Build a labeled fixture corpus** (30–50 real, anonymized page snapshots: true positives across major ATS/job boards, true negatives — homepages, contact forms, blog posts) and unit-test `detect()` against it. This turns the 0.4 threshold from a guess into a tuned, regression-tested value.
4. **Add a same-tab detection cache invalidation on URL change** even without a full document reload, so a stale "JOB" badge doesn't linger after the user navigates away within an SPA.

5. **Defer a WASM/ML classifier** — six weighted signals already provide good coverage; add complexity only if the fixture corpus in (3) reveals a real accuracy gap Tiers 1–2 can't close.

5. Simplification Opportunities

Area	Observation	Recommendation
Duplicate JSON-LD traversal	<code>detect.js</code> 's <code>hasJobPostingJsonLd()</code> and <code>scrape.js</code> 's <code>collectJobPostings()</code> both iterate <code>script[type="application/ld+json"]</code> and both walk the same <code>@graph</code> /array-or-object shape independently, in two different files, with two slightly different traversal implementations.	Extract one shared <code>src/content/jsonld.js</code> helper (parse-once, return all <code>JobPosting</code> nodes) that both <code>detect.js</code> and <code>scrape.js</code> import. Removes duplicated parsing logic and the risk of the two implementations silently diverging.
Two separate keyword lists doing similar jobs	<code>JOB_HEADING_KEYWORDS</code> is reused for both heading detection <i>and</i> title/meta-tag detection (<code>hasJobMetaTags</code>) — reasonable reuse, not a problem. <code>JOB_BODY_KEYWORDS</code> is a separate, unrelated list. Fine as-is; flagging only because it's easy to assume these should merge — they shouldn't (different precision/recall needs).	No change — documenting this to save future engineers time re-deriving why there are two lists.
Site handling is entirely undifferentiated	One heuristic set for every domain is simple today, but "simple" here also means "no place to encode ATS-specific knowledge when the generic heuristics fail on a specific site." This is the right trade-off <i>while</i> there are zero adapters — adding a whole adapter-registry abstraction now (per the TDD's <code>ATSAdapter</code> interface) would be premature: there's no evidence yet of which sites need it.	Don't build the adapter registry speculatively. Instead, add an escape hatch: a small, flat, url-pattern-keyed override map (<code>{ pattern, contentSelector }</code>) that <code>pickMainContent()</code> consults first if present. Promote to a full adapter interface only once 3+ real sites need one.
<code>CONTENT_SELECTORS</code> ordering implies priority but scoring ignores it	The list is commented "in rough priority" but <code>pickMainContent()</code> scores every match from every selector and takes the global max — the list order has zero effect on the outcome, which is misleading to read.	Either remove the "priority order" comment (score-based selection makes ordering cosmetic), or make ordering actually matter with a small per-position weight if empirical testing shows selector priority should break scoring ties.
Configuration is scattered	<code>MAX_CHARS</code> , <code>MIN_JSONLD_DESCRIPTION</code> , <code>JOB_BODY_THRESHOLD</code> , the 0.4 detection threshold, and the 500-element scoring cap are each a local constant in their own file with no single place to see/tune them together.	Low priority — these are genuinely local concerns, not shared config. Only worth centralizing if/when a config UI or A/B-testable thresholds become a real requirement; don't add an abstraction for a problem that doesn't exist yet.

6. Reliability Improvements

Improvement	Advantage	Disadvantage
MutationObserver-based re-scoring (detection + extraction)	Directly fixes the highest-frequency failure (§3): SPA/async-rendered content. Debounced, so cost is bounded.	Adds statefulness to content scripts (currently pure/stateless) — needs careful teardown to avoid observer leaks across re-injections; needs a hard timeout so a page with continuous mutation (ads, chat widgets) doesn't observe forever.
SPA navigation detection (<code>pushState</code> / <code>popstate</code> hook)	Fixes stale badge/detection on client-side route changes without waiting for a full reload.	Patching <code>history.pushState</code> from a content script (isolated world) only sees <code>pushState</code> calls made through the same wrapped reference if injected before the page's own router runs — timing-sensitive; a <code>popstate</code> -only listener is safer but misses forward navigations that don't emit it.
Confidence-gated extraction quality signal	<code>scrapePage()</code> already returns <code>source</code> (" <code>json-ld</code> " / " <code>readable</code> " / " <code>body</code> ") as a provenance signal — cheap to extend into an explicit quality score (e.g., penalize " <code>body</code> " fallback, penalize very short text) surfaced to the backend so low-confidence extractions can be flagged for the user rather than silently analyzed.	Requires backend awareness of the new field (coordinate with the backend owner) and a UI decision on what "low confidence, proceed anyway?" looks like.
Retry mechanism on SCRAPE_PAGE	Side panel currently treats a failed <code>tabs.sendMessage</code> as terminal ("reload the tab"). A short retry (e.g., 2 attempts, 300ms apart) covers the common race where the content script hasn't finished injecting yet on a freshly-loaded tab.	Marginal — most failures here are permanent (chrome:// pages, no content script match) rather than transient, so retry helps a narrow slice of cases.
Structured data extraction before DOM scraping	Already implemented and is the single best-value design decision in the current system — keep it, don't regress it.	N/A — already done.
Network/API interception (watch <code>fetch</code> /XHR for a JSON job payload the page itself loaded)	Would solve the async-content problem at the source for sites that fetch job data as JSON (rather than waiting for it to paint into the DOM) — often cleaner/more complete than scraping rendered text.	Requires a <code>MAIN</code> -world injected script (content scripts run in an isolated world and can't see the page's own <code>fetch</code> calls without extra injection); heuristics for "which network response is the job JSON" are themselves fragile and per-site. Higher effort than MutationObserver for similar payoff — worth prototyping only after the observer-based fix ships and if gaps remain.
Site-specific extraction modules	Would raise accuracy ceiling on specific problem sites once identified.	Premature without evidence of which sites actually need it (see §5) — avoid building the registry speculatively.
Validation of extracted content	A cheap floor check (e.g., reject/flag extractions under ~150 chars, or with zero job-keyword overlap) would catch silent " <code>body</code> fallback returned nav junk" cases before they reach the backend/LLM.	Risk of false negatives on genuinely short but valid postings (rare, but possible) — should flag for review, not hard-block.

7. Testing Strategy

7.1 Unit Tests

Component	Current coverage	Gap to close
scrape.js	Good — <code>tests/scrape.test.mjs</code> covers JSON-LD extraction, multi-posting selection, noise stripping, hidden-content skip, truncation, and the return-shape contract.	Add: shadow-DOM traversal (once built), malformed/edge-case JSON-LD (circular <code>@graph</code> , non-object array entries), extremely link-dense containers (link-density edge of the scoring formula).
detect.js	None. No test file exists.	Highest-priority gap. Unit-test each tier function in isolation (<code>urlScore</code> , <code>hasJobPostingJsonLd</code> , <code>hasApplicationForm</code> , <code>hasJobHeadings</code> , <code>hasJobBodyContent</code> , <code>hasJobMetaTags</code>) plus the combined <code>detect()</code> confidence math against a labeled fixture set (true positives + known false-positive shapes like newsletter/contact forms).
background/index.js (message router)	None.	Test the router's <code>switch</code> dispatch (each message type calls the right handler, unknown type returns <code>{ok: false}</code>), and the tab-cleanup listener.
background/api.js	None.	Mock <code>fetch</code> ; test URL construction, error-shape normalization (non-2xx → <code>{ok: false, error}</code>), and the base-URL fallback/trim logic.
background/storage.js	None.	Straightforward — mock <code>chrome.storage.session</code> , verify per-tab key prefixing and remove-on-miss doesn't throw.

7.2 Integration Tests

- End-to-end message flow: content script `PAGE_DETECTED` → service worker persists → badge set → side panel reads via `GET_DETECTION` and renders the correct signal grid. (Currently only exercised manually.)
- Full scrape round-trip: side panel `SCRAPE_PAGE` → content script extraction → `ANALYZE_JOB` → service worker `fetch` → rendered analysis JSON, run against a mocked backend so this doesn't depend on the live Python service.
- Tab lifecycle: open tab → detect → close tab → verify `chrome.storage.session` keys are removed (`tabs.onRemoved` handler).
- A fixture-replay suite: capture real (anonymized) HTML snapshots from the top ~10 target sites/ATS platforms and replay them through `detect()` + `scrapePage()` in CI on every change to `content/*` — this is the single highest-leverage integration test to add, since it catches regressions before they reach a real browser.

7.3 Manual Testing Checklist

Job boards / ATS (golden path)

- LinkedIn Jobs (SPA — verify detection survives in-app navigation between postings)
- Indeed (`indeed.com/viewjob`)
- Glassdoor job listing
- Greenhouse-hosted posting (`boards.greenhouse.io`)
- Lever-hosted posting (`jobs.lever.co`, including any iframe-embedded variant)

- Workday (*.myworkdayjobs.com — multi-step SPA)
- iCIMS, SmartRecruiters, Ashby, Workable, BambooHR, Taleo, SuccessFactors postings
- A generic company careers page with no ATS (custom-built site, no JSON-LD)

Edge cases

- Job posting behind a cookie-consent overlay (verify text extraction ignores the banner)
- Page with multiple JSON-LD blocks, only one of which is a JobPosting
- Page with a JobPosting JSON-LD block containing a stale/short description (verify fallback to readable extraction per MIN_JSONLD_DESCRIPTION)
- Very long job description (>30k chars) — verify truncation flag and word-boundary cut
- Non-English job posting (verify detection still fires via URL/JSON-LD tiers even if body keywords don't match)
- Job page opened, then navigated (same tab, full reload) to a non-job page — verify badge clears
- Job page opened, then navigated (same tab, client-side/SPA) to a different posting — verify detection/scrape refresh (currently expected to **fail** — see §3)
- chrome:// or extension-gallery page (no content script) — verify side panel shows "Can't read this page" rather than throwing
- Backend unreachable — verify health check and analyze/generate calls degrade to a visible error, not a hang

8. Performance Review

Aspect	Assessment
DOM traversal efficiency	Reasonable: <code>pickMainContent()</code> caps scoring at 500 elements; <code>TreeWalker</code> with <code>FILTER_REJECT</code> prunes whole noise subtrees instead of visiting every descendant. <code>scoreElement()</code> cheaply pre-filters on <code>textContent.length < 200</code> before the more expensive <code>innerText</code> read (which forces layout).
Memory usage	Low — no cloning, no retained DOM references beyond the synchronous call; results are plain JSON. Nothing to indicate a leak risk in the current stateless design.
Duplicate processing	Present but minor: <code>detect.js</code> and <code>scrape.js</code> each independently parse JSON-LD scripts (§5) — on a page with many <code><script type="application/ld+json"></code> blocks this is double the JSON parsing work, though in practice these blocks are few and small.
Unnecessary queries	<code>extractText()</code> builds a fresh <code>noise</code> Set via <code>querySelectorAll</code> for every one of the ~15 <code>NOISE_SELECTORS</code> on every call — acceptable for a single on-demand scrape, but would compound if re-run frequently by a future <code>MutationObserver</code> without debouncing.
Large pages	Handled: 500-element scoring cap and 30k-char truncation both exist specifically to bound cost on large pages.
Extension startup time	Content scripts are three small, dependency-free plain-JS files (no bundler, no framework) injected at <code>document_idle</code> — negligible startup cost. Service worker similarly has no heavy init.
Opportunities	If a <code>MutationObserver</code> is added (§6), debounce it (300–500ms) and cap total observation time (~3s) so it can't run indefinitely on pages with continuous background DOM churn (chat widgets, ad refreshes, analytics beacons appending nodes).

9. Final Recommendations — Prioritized Roadmap

Quick Wins (1–2 days each)

Item	Impact	Effort	Priority
Unit-test <code>detect.js</code> against a small labeled fixture set (true positives + common false-positive shapes)	Closes the biggest test-coverage gap; makes the 0.4 threshold a tuned value instead of a guess	1–2 days	High
Add <code>all_frames: true</code> + per-frame detection/scrape reporting	Fixes iframe-embedded Lever/Workday postings	1–2 days	Medium
Extract shared JSON-LD parsing helper (dedupe <code>detect.js</code> / <code>scrape.js</code>)	Removes duplicate logic, prevents future divergence	<1 day	Low
Add a floor validation on extracted text (min length / keyword sanity) before sending to backend	Catches silent "body fallback returned junk" cases	1 day	Medium
Add <code>fetch timeout</code> / <code>AbortController</code> to <code>background/api.js</code>	Prevents indefinite hangs when the backend is unresponsive	<1 day	Medium

Medium Improvements (1–2 weeks)

Item	Impact	Effort	Priority
Debounced <code>MutationObserver</code> re-scoring for both detection and extraction	Fixes the single largest real-world failure mode: async/SPA-rendered job content	1–2 weeks	Highest
<code>pushState</code> / <code>popstate</code> re-detection on SPA route changes	Fixes stale badge/detection on client-side navigation (LinkedIn, embedded SPA careers pages)	3–5 days	High
Fixture-replay integration suite (10+ real, anonymized ATS snapshots in CI)	Catches extraction/detection regressions before they reach production, matching the TDD's own CI intent (\$5.14)	1–2 weeks	High
URL-pattern-keyed content-selector override map (escape hatch, not a full adapter system)	Lets a specific problem site be fixed without touching shared heuristics	3–5 days	Medium
Shadow-DOM-aware traversal for both selector queries and <code>TreeWalker</code>	Future-proofs against growing use of web components in ATS UIs	1 week	Low–Medium

Larger Architectural Improvements

Item	Impact	Effort	Priority
Network/API interception (MAIN-world script watching <code>fetch</code> /XHR for job JSON payloads)	Solves async-content extraction at the source for sites that load job data as JSON; likely more complete than DOM scraping for those sites	2–4 weeks	Medium — prototype only after <code>MutationObserver</code> ships, if gaps remain
Full per-ATS adapter registry (the TDD's <code>ATSAdapter</code> interface)	Would raise the accuracy ceiling for specific known-problem platforms	3–6 weeks	Low today — build only once the override map (medium tier) proves 3+ sites need dedicated logic; premature otherwise
Confidence-scored extraction quality contract with the backend	Lets the backend/UI gate low-quality extractions before spending	2–3 weeks	Medium — needs backend-owner coordination since it's a cross-repo

(shared schema for "how much should we trust this text")	an LLM call on them		contract change
--	---------------------	--	-----------------

Scope note: Autofill, per-field form-filling, and submission detection (TDD §6.3, 6.6–6.8) are not included in this roadmap because they don't exist in the codebase today — they are a distinct, larger product surface (form reading/writing) built on top of, but separate from, the detection/extraction pipeline reviewed here. They should get their own dedicated review before implementation begins.